PIDray Official Repository: Deep Architecture, Configuration, and Reproduction Guide

This document is the implementation companion to the PIDray theory review. It systematically explains repository ancestry, exact source files, tensor flow, released model components, hyperparameters, data-loader modifications, commands, paper–code differences, and reproduction acceptance tests. The reviewed official commit is `2acc524`.

1. Repository Identity and Upstream Lineage

The released journal detector is not a class named `PIDrayNet`. It is a `CascadeRCNN` assembled by `configs/cascade_mask_rcnn_r101_with_R0R1.py` inside a modified, vendored MMDetection 2.25.0 tree. PIDray-specific behavior is distributed across the custom AttFPN, the generic two-stage detector, a confidence-transformer directory, positional encoding, and dataset initialization.

Table 1: Repository ancestry and ownership of functionality.

Source	Version / location	What it supplies
PIDray official	`lutao2021/PIDray`	Configs, AttFPN, R0 integration, loader changes, dataset/checkpoint links and scripts.
MMDetection	Vendored 2.25.0	Cascade Mask R-CNN, RPN, RoIAlign/heads, mask targets, COCO datasets/evaluator, runner and registries.
MMCV	1.5.2	Convolution wrappers, NMS, distributed runner utilities, `ContextBlock`.
torchvision	0.11.0	ImageNet ResNet-101 initialization.
Query2Label / DETR	Local transformer header and design lineage	Learned-query cross-attention with decoder self-attention removed by default.
COCO API	2.0.2	Box/mask AP and AR evaluation.

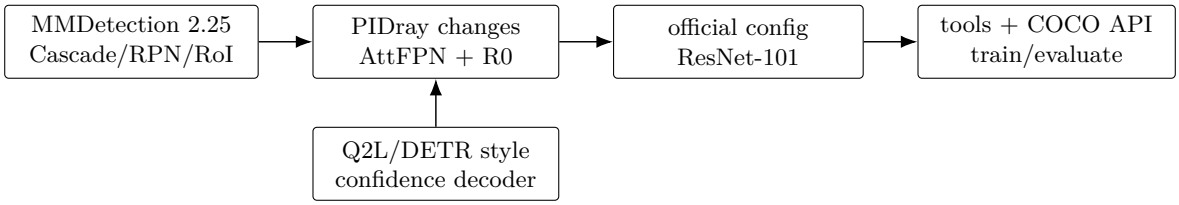


Figure 1: Implementation lineage. The official model is a configuration plus modifications to the vendored framework.

2. Configuration Resolution

The top-level config is self-contained rather than composed from many base files. It declares model, dataset pipelines, training/testing rules, optimization, evaluation, distributed settings and paths. The correct build sequence is:

$$\texttt{config} \rightarrow \texttt{CascadeRCNN} \rightarrow \{\texttt{ResNet}, \texttt{AttFPN}, \texttt{RPNHead}, \texttt{CascadeRoIHead}\}.$$

The inherited `CascadeRCNN` ultimately derives from the modified `TwoStageDetector`, where R0 is created for every two-stage detector using that base. Thus R0 is not declared as a normal config submodule; searching only the config will miss it.

Table 2: Exact source map in recommended reading order.

Order	Path	What to verify
1	`README.md`	Dataset/checkpoint locations and command templates.
2	`configs/cascade_mask_rcnn_r101_with_R0R1.py`	Complete model, pipelines, optimizer, schedule and test settings.
3	`mmdet/models/necks/attfpn.py`	Standard FPN, five-level resize, channel/spatial softmax, context block and residual output.
4	`mmdet/models/detectors/two_stage.py`	R0 construction, labels, BCE, selection, gradient scaling and score suppression.
5	`mmdet/models/confidence_branch/transformer.py`	Two-layer learned-query decoder and removed self-attention.
6	`mmdet/models/utils/positional_encoding.py`	Normalized 2-D sine/cosine position.
7	`mmdet/datasets/custom.py`	Disabled empty-image filtering.
8	`tools/dist_train.sh`, `tools/dist_test.sh`	Distributed launch and evaluation entry points.
9	`requirements.txt`, `mmdet/version.py`	Intended legacy environment and inconsistent torchaudio line.

3. Exact Tensor Path

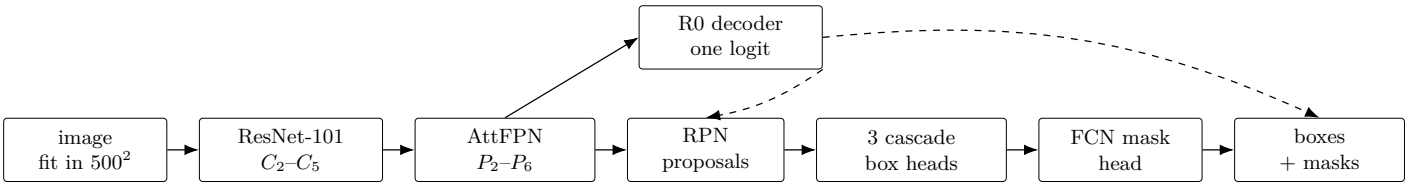


Figure 2: Released code path. Dashed arrows denote R0 control/suppression, not feature tensors.

3.1 Preprocessing and backbone hyperparameters

Table 3: Input and ResNet-101 configuration.

Parameter	Released value	Effect
Resize	`img_scale=(500,500)`, keep ratio	Fits inside 500 square; one dimension may be smaller.
Augmentation	Horizontal flip probability 0.5	Training only; no multi-scale test.
Normalization	ImageNet mean/std; BGR-to-RGB	Matches torchvision initialization.
Padding	Divisor 32	Batch tensor dimensions divisible by pyramid stride.
Backbone	ResNet-101, stages $[3, 4, 23, 3]$	Outputs 256/512/1,024/2,048 channels.
Output strides	4/8/16/32	C_2–C_5.
Frozen stages	1	Stem and first residual stage frozen.
Normalization	BN parameters declared trainable; `norm_eval=True`	BatchNorm running behavior remains in eval mode.
Initialization	`torchvision://resnet101`	ImageNet weights.

3.2 AttFPN implementation parameters

The neck builds normal FPN outputs and then calls five separate `selective_attention` modules, one per target level. Every source is resized to the target grid. The channel branch uses $M = 5$, bottleneck width $d = 128$, global average pooling and level-wise softmax. The spatial branch uses channel mean/max, five 7×7 convolutions and level-wise softmax. The two weighted sums are added, passed through `ContextBlock(256,1/16)`, and added to the original target feature.

Table 4: AttFPN tensor and operator contract.

Element	Value	Implementation detail
Inputs	256/512/1,024/2,048 channels	Four ResNet stages.
Base outputs	Five maps, each 256 channels	Strides 4, 8, 16, 32, 64.
Fine-to-coarse resize	Adaptive max-pool	Used when source has finer grid than target.
Coarse-to-fine resize	Nearest interpolation	Also applied to equal level in code.
Channel attention	$256 \to 128 \to 5 \times 256$	Softmax dimension is source-level dimension.
Spatial attention	mean/max $\to$ 2 channels; five 7×7 convs	Per-location source-level softmax.
Dependency refinement	Context block ratio 1/16	Long-range/global context.
Output	refinement + original target	Five independent residual outputs.

4. R0 Code Semantics

R0 uses refined P_6. The constructor hard-codes hidden width 256, dropout 0.1, 4 heads, FFN 1,024, no encoder, 2 decoder layers, no decoder self-attention, one learned query, normalized sine position and a $256 \to 1$ linear head. BCE is multiplied by 0.1.

Table 5: Training and inference behavior implemented in the modified two-stage detector.

Behavior	Exact code consequence
Binary target	1 if `len(gt_bboxes[i])>0`; otherwise 0.
Selection	Detached sigmoid probability > 0.5; detection loss cannot train the threshold.
Selected batch	Feature levels, image, metadata, boxes, labels and masks are subset.
No selected image	Full batch still runs through detector, then fine-detector gradient factor is 0.
Loss display	Stop-gradient expression preserves printed detector loss while scaling its gradient by selected fraction.
Inference	RPN and RoI execute first; low R0 then zeroes scores for the first batch element.
Batch assumption	Final score suppression indexes only `confidence_score[0]`.
Efficiency claim	Released code is not a true early exit; any early-exit patch is a new variant.

5. RPN, Cascade, Mask, and Test Hyperparameters

Table 6: Complete region and mask settings from the official config.

Component	Values	Notes
RPN conv	input 256; feature 256	Shared across five levels.
Anchors	scale 8; ratios 0.5/1/2; strides 4/8/16/32/64	Three anchors/location; ratio-1 sizes 32–512.
RPN coder/loss	means 0; stds 1; sigmoid CE; SmoothL1 $\beta = 1/9$	Class-agnostic.
RPN assignment	pos 0.7; neg 0.3; min pos 0.3; low-quality matching	Sample 256; positive fraction 0.5.
Train proposals	pre-NMS 2,000; max 2,000; NMS 0.7	No minimum size.
Test proposals	pre-NMS 1,000; max 1,000; NMS 0.7	No minimum size.
Box RoI	RoIAlign 7; sampling ratio 0; strides 4/8/16/32	Two FC layers, width 1,024.
Cascade stages	IoU 0.5/0.6/0.7; weights 1/0.5/0.25	Each samples 512 RoIs; positive fraction 0.25.
Box coder stds	(0.1,0.1,0.2,0.2); (0.05,0.05,0.1,0.1); (0.033,0.033,0.067,0.067)	Class-agnostic regression; 12 classes.
Mask RoI/head	RoIAlign 14; four 256-channel convs; 12 outputs	Target size 28; class-specific masks.
Final test	score 0.05; NMS 0.5; max 100; mask threshold 0.5	Box and segmentation COCO metrics.

6. Optimizer, Schedule, Runtime, and Paths

Table 7: All released training/runtime values relevant to reproduction.

Item	Released value
Batch/loading	4 samples/GPU; 4 workers/GPU; config GPU IDs 0–3; effective batch 16.
Optimizer	SGD, LR 0.02, momentum 0.9, weight decay 0.0001; no gradient clipping.
Warm-up	Linear, 500 iterations, start ratio 0.001.
LR schedule	Step policy; drops at epochs 8 and 11.
Runner	Epoch-based, 12 epochs; checkpoint each epoch; log every 50 iterations.
Evaluation	`bbox` and `segm`; configured at epoch 12.
Automatic LR scaling	Disabled; declared base batch size 16.
Multiprocessing	NCCL; fork; OpenCV threads 0.
Train paths	`annotations/train.json`, `train/`.
Val/test paths	Both use `annotations/test.json`, `test/`.

Environment caution. The requirements evidence points to PyTorch 1.10.0 + CUDA 11.1, torchvision 0.11.0, `mmcv-full` 1.5.2, vendored MMDetection 2.25.0, pycocotools 2.0.2 and NumPy 1.22.4. The same file also names a ROCm torchaudio wheel, which is inconsistent with the CUDA stack and unnecessary for detection. Install only packages actually imported.

7. Dataset Loader Modification

Standard MMDetection 2.x can remove empty-target images when empty filtering is enabled. PIDray comments out the `_filter_imgs()` call in the local custom dataset initialization, so normal baggage images remain. This is a scientifically essential repository modification, not an incidental convenience: removing normal images destroys negative R0 supervision.

Table 8: Data assertions required before training.

Assertion	Failure prevented
Post-loader image count matches manifest	Silent image filtering.
Normal-image count remains nonzero and expected	Loss of R0 negative supervision.
Twelve class IDs/names match checkpoint order	Label permutation.
Boxes are valid and inside image	Proposal/evaluator corruption.
Polygons decode and overlay correctly	Invalid mask AP.
Easy/Hard/Hidden ID manifests are disjoint	Subset leakage or duplicate evaluation.

8. Commands and Reproduction Procedure

The official command forms are:

```
sh tools/dist_train.sh CONFIG N
sh tools/dist_test.sh CONFIG CHECKPOINT N -out RESULT -eval bbox segm
```

They are entry patterns, not a complete experiment record. The process count, visible GPUs, config hash, checkpoint hash, seed, data hashes and environment lock must be recorded separately.

1. Pin the official commit and build the legacy environment. Confirm `mmdet.__file__` points inside the clone.
2. Create a PIDray manifest: hashes, class order, positive/normal counts, subset IDs and rendered samples.
3. Copy the official config and change only paths/work directory.
4. Evaluate released weights before training. Save raw predictions and Easy/Hard/Hidden metrics.
5. Smoke train a subset containing positive and normal images. Log R0 labels/logits, selected fraction, detector gradient norms, RPN recall and mask outputs.
6. Create validation only from training. Never use the official test subsets for tuning.
7. Reproduce the 12-epoch run with effective batch 16 where possible.
8. Freeze the protocol, then evaluate final test subsets and gate metrics.

9. Paper–Code Reconciliation and Known Risks

Table 9: Issues that must be reported rather than silently corrected.

Issue	Required treatment
R0 described as pre-screening	Released inference suppresses after detector execution; do not claim speedup.
Fine loss described as scaled	Code preserves printed value and scales only gradient.
Val/test alias	Create train-derived validation; do not tune on official test.
Batch-one score suppression	Use exact batch-one inference or label a batch-safe patch.
Normal-image filtering	Use local loader; assert post-loader counts.
Paper hardware vs README command	Record actual GPU count, per-GPU/global batch and LR.
Model called PIDrayNet	Cite the config and components; no such class exists.
Framework modernization	A port to MMDetection 3.x is a new implementation after exact reproduction.

10. Instrumentation for the Proposed Study

Before adding a background-confusion component, log: normal-image false boxes per image, R0 false-negative/false-positive rates, RPN recall by item size and subset, high-confidence unmatched RoIs, mask leakage outside target boundaries, and latency by backbone/R0/RPN/RoI. The primary ablation should keep R0 unchanged while adding only one proposal- or instance-level mechanism. This preserves a contribution distinct from the existing image-level gate.

11. Verified Links

- Official repository: `github.com/lutao2021/PIDray`.
- Journal paper: arXiv:2211.10763.
- Upstream: MMDetection 2.25.0; MMCV 1.5.2; Query2Label.